Project Documentation: Phase 3 - Node.js & MongoDB Integration

Project: IoT DDoS Detection Dashboard

Phase: 3 (Backend Integration)

1. The Goal of Phase 3

The goal of Phase 3 is to build a "Traffic Cop" (Node.js) that takes incoming network data from a user, securely asks your Python ML model (Flask) for a prediction, and then permanently saves that result in a cloud database (MongoDB). This upgrades your project from a standalone mathematical script into a real-world, data-storing web application.

2. Step-by-Step Explanation

Step 1: Creating the Backend Folder

What we are doing: Creating a brand new, empty folder specifically for Phase 3.

Why we are doing it: We must keep our Node.js (JavaScript) environment completely separate from our Flask (Python) environment.

Step 2: Setting up Node.js

What we are doing: Running npm init -y to initialize Node.js, which creates package.json.

Why we are doing it: package.json keeps track of dependencies.

Step 3: Installing Packages

We install express, axios, mongoose, and cors.

Why: Express = server, Axios = communication, Mongoose = MongoDB connection, CORS = security.

Step 4: Creating the Server (server.js)

Main backend file handling all logic.

Step 5: Connecting to Flask API

Send data to http://127.0.0.1:5000/predict for ML prediction.

Step 6: Connecting to MongoDB Atlas

Store predictions permanently in cloud database.

Step 7: Creating a Schema

Define structure for safe and valid data storage.

Step 8: Creating API Endpoints

POST /data → send data, get prediction, save it.

GET /data → fetch all stored history.

3. The Full System Flow

Incoming Data → Node.js → Flask → Prediction → MongoDB → Response to User

4. Final Summary

You now have a microservices architecture with Node.js handling backend and Flask handling ML.